SWIGLU AS A LOW-RANK TENSOR MODEL

ANIKET DESHPANDE

In this brief note, I explore a tensor-theoretic interpretation of the transformer FFN, viewing the SwiGLU block as a highly structured low-rank interaction layer over the residual stream. By isolating its first nontrivial multiplicative term, I show that SwiGLU induces a CP-factorized second-order interaction tensor with an implicit diagonal core, which in turn suggests a principled generalization via a learned low-rank Tucker core that allows the model to learn which latent features should interact multiplicatively.

note: this idea is in *very* early stages. I'm actively working on more concrete theory and experiment design and will update the note as I make progress. if you have access to this note and wish to discuss ideas/fixes/implementation, please reach out!

We begin with the residual stream. At a fixed token position, let $x \in \mathbb{R}^d$ denote the residual stream vector entering the feedforward network (FFN) block. In a modern transformer [Vas+17], attention is responsible for moving information across tokens, while the FFN acts pointwise on each token's residual representation. In the architectures of relevance, this pointwise map is usually a gated MLP of the SwiGLU form [Sha20]

$$y = U^T h, \quad h = \left(W^T x\right) \odot \sigma\left(G^T x\right), \tag{1}$$

where $W, G \in \mathbb{R}^{d\times m}$ and $U \in \mathbb{R}^{m\times d}$. Here, W is the main up-projection, G is the gate projection, and U is the down-projection, and $h \in \mathbb{R}^m$ is the hidden activation. Writing the j-th columns of W and G as $w_j, g_j \in \mathbb{R}^d$, the j-th hidden coordinate is

$$h_j(x) = \left(w_j^\top x\right) \sigma\left(g_j^\top x\right). \tag{2}$$

Already, we can see a core structural difference between the FFN and the attention map. From the kernel-regression viewpoint, it is Nadaraya-Watson regression with a Gaussian kernel [Nad64, Pai25]. From a statistical mechanical viewpoint, the attention weights form a Boltzmann distribution over key-value pairs,

$$\alpha_i(q) = \frac{\exp\left(\beta q^T k_i\right)}{\sum_j \exp\left(\beta q^T k_j\right)},$$

with inverse temperature $\beta = \tau^{-1}$. The same mechanism therefore interpolates continuously between diffuse averaging at high temperature and sharp retrieval at low temperature. This dual interpretation is part of attention's tractability. The FFN, specifically the SwiGLU architecture, has no comparable theory. Yet it contains the majority of the model's parameters and is plausibly where many of the remaining gains could lie. I'd like to analyze the FFN from a tensor perspective.

Going back to (2), each unit computes a two-feature interaction. It is important to ask, what *kind* of interaction is it, and what structural assumptions does the layer impose on the space of possible feature interactions?

For SwiGLU, the key point is that the nonlinearity already has an exact multiplicative factorization. Since

$$\mathrm{SiLU}(z) = z\sigma(z), \quad \sigma(z) = \frac{1}{1+e^{-z}}, \tag{3}$$

we may rewrite the j-th hidden unit exactly as

$$h_j(x) = (w_j^\top x)\mathrm{SiLU}(g_j^\top x) = (w_j^\top x)(g_j^\top x)\sigma(g_j^\top x). \tag{4}$$

Thus each hidden unit is an exact bilinear interaction between two learned scalar projections, modulated by an input-dependent scalar gate. The quadratic part is therefore an exact structural component of the unit for every input.

The factor

$$(w_j^\top x)(g_j^\top x) = x^\top (w_j g_j^\top) x$$

has a clean algebraic interpretation. Because $w_j g_j^\top$ is an outer product, it is a matrix of rank one. So each hidden unit builds a multiplicative interaction out of a rank-one separable interaction between two learned directions in residual space, together with a scalar routing coefficient $\sigma(g_j^\top x)$. Each hidden unit sees two directions in residual space, projects onto them, then multiplies those two scalar responses, and then modulates the strength of that interaction by a gate. This is a very specific inductive bias. The model is not learning an arbitrary interaction pattern among latent features; it is learning a diagonal pairing of two projected feature sets.

The tensor viewpoint makes this more explicit. Define the quadratic feature tensor

$$x \otimes x \in \mathbb{R}^{d\times d}, \quad (x\otimes x)_{ab} = x_a x_b.$$

A fully general second-order hidden layer with m interaction channels would require a third-order tensor $\mathcal{T} \in \mathbb{R}^{m\times d\times d}$ with hidden coordinates

$$h_j(x) = \sum_{a=1}^{d}\sum_{b=1}^{d} \mathcal{T}_{jab} x_a x_b = \langle \mathcal{T}_j, x\otimes x\rangle. \tag{5}$$

However, SwiGLU does not allow arbitrary slices $\mathcal{T}_{j,:,:}$. Instead, by (4)

$$h_j(x) = \sigma(g_j^\top x)\langle w_j \otimes g_j, x\otimes x\rangle = \sigma(g_j^\top x)(w_j^\top x)(g_j^\top x). \tag{6}$$

So every hidden slice is forced to factor as an outer product, and the nonlinearity acts only through an input-dependent scalar coefficient. Now, we include the output projection. The output becomes

$$y_o = \sum_{j=1}^{m} U_{jo} h_j(x) = \sum_{j=1}^{m} U_{jo}\sigma(g_j^\top x)(w_j^\top x)(g_j^\top x) \tag{7}$$

$$= \sum_{a=1}^{d}\sum_{b=1}^{d}\left(\sum_{j=1}^{m} U_{jo}\sigma(g_j^\top x) w_{ja} g_{jb}\right) x_a x_b. \tag{8}$$

We therefore define the exact input-dependent interaction tensor $\mathcal{A} \in \mathbb{R}^{d\times d\times d}$

$$y_o = \sum_{a,b} \mathcal{A}_{oab}(x) x_a x_b, \quad \mathcal{A}_{oab}(x) = \sum_{j=1}^{m} U_{jo}\sigma(g_j^\top x) W_{aj} G_{bj}. \tag{9}$$

This is precisely a canonical polyadic (CP) decomposition [KB09] of the interaction tensor

$$\mathcal{A}(x) = \left[\!\left[U^\top \mathrm{diag}(\alpha(x)), W, G\right]\!\right] = \sum_{j=1}^{m} \alpha_j(x) u_j \otimes w_j \otimes g_j, \quad \alpha_j(x) = \sigma(g_j^\top x), \tag{10}$$

where u_j is the output factor associated with the j-th hidden unit. So, for every input x, SwiGLU realizes an exact input-routed family of low-rank CP interaction tensors.

SwiGLU realizes multiplicative feature mixing [JSSB20], but only through sums of rank-one separable factors. Even more strongly, the routing is coordinate-wise in the hidden space: the j-th feature in one branch is only allowed to interact with the j-th feature in the other branch, and the

resulting rank-one interaction atom $u_j \otimes w_j \otimes g_j$ is gated only by the scalar sigmoid $\sigma(g_j^\top x)$ associated with that channel. This is a useful bias; it gives multiplicative structure at low cost. However, it is also a strong restriction. The model is not learning an arbitrary interaction pattern among latent features; it is learning a diagonal pairing of two projected feature sets, where only same-index latent features are allowed to interact. In tensor speak, SwiGLU imposes a superdiagonal interaction core in latent space.

This naturally suggests a more principled generalization. Instead of forcing the interaction routing to be same-index in hidden space, we introduce a learned low-rank core tensor. The simplest way to do this is through a Tucker decomposition. We choose factor matrices $P, Q \in \mathbb{R}^{d\times r}$ and $R \in \mathbb{R}^{d\times s}$, and a core tensor $\mathcal{C} \in \mathbb{R}^{s\times r\times r}$. We project the residual stream into two latent feature spaces $p = P^T x \in \mathbb{R}^r$ and $q = Q^T x \in \mathbb{R}^r$, then define interaction channels as

$$z_\alpha = \sum_{i=1}^{r}\sum_{j=1}^{r} \mathcal{C}_{\alpha ij} p_i \mathrm{SiLU}(q_j), \quad \alpha = 1, \ldots, s, \tag{11}$$

and map them back to residual space with $y = Rz$. This is a Tucker-factorized generalization of SwiGLU. In SwiGLU, each projected feature p_i can only interact with the corresponding gated feature q_i. In the Tucker formalism, the learned core $\mathcal{C}$ decides which projected features should interact multiplicatively. Same-index interactions become a special case when $\mathcal{C}$ is superdiagonal.

Using again the identity $\mathrm{SiLU}(z) = z\sigma(z)$, we can equivalently write the interaction channels as

$$z_\alpha = \sum_{i=1}^{r}\sum_{j=1}^{r} \mathcal{C}_{\alpha ij} \sigma(q_j) p_i q_j, \tag{12}$$

so this generalized block still realizes an input-dependent interaction tensor, but now with a learned low-rank cross-channel core rather than same-index routing only. This generalization allows the model to learn richer interaction patterns among latent features, while still maintaining a low-rank structure that keeps the parameter count manageable.

We can directly recover SwiGLU if $r = s = m$ and $\mathcal{C}_{\alpha ij} = \delta_{\alpha i}\delta_{ij}$. Then, $z_i = p_i \mathrm{SiLU}(q_i)$, which is exactly the original elementwise gating pattern. So Tucker strictly generalizes SwiGLU rather than replacing it with an unrelated construction.

This direction is promising for three main reasons. First, it is driven by a principled analysis that characterizes the exact interaction structure of the FFN, rather than a local approximation whose validity depends on the operating regime. Second, it introduces a clean tradeoff between expressivity and compute via r and s. The latent rank r and the number of interaction channels s control how much multiplicative structure the block can represent. Third, the contraction should remain reasonably hardware-friendly, since it can be implemented through batched matrix multiplications and small tensor contractions rather than an unstructured dense higher-order tensor.

There is also an interesting interpretability motivation. One reason attention has been unusually amenable to mechanistic analysis, and why such analysis has already enabled principled architecture design [Ols+22, Fu+23], is that its structure is additive and probabilistic: attention heads decompose into weights over values, and those weights often admit clear semantic or algorithmic roles. The FFN block has not yielded to the same kind of decomposition. One plausible reason is that its interaction geometry is much harder to factor cleanly downstream. But the exact representation above suggests that SwiGLU already decomposes into a dictionary of fixed rank-one interaction atoms $u_j \otimes w_j \otimes g_j$ together with input-dependent routing coefficients $\alpha_j(x) = \sigma(g_j^\top x)$. This separates what interaction is available from when that interaction is used.

If the multiplicative structure inside the FFN were made more explicit and low-rank, then sparse decomposition methods (dictionary methods, sparse autoencoders, transcoders) [Bri+23] might have a cleaner object to work with. Rather than trying to recover structure only from downstream

activations, one could try to analyze the routed interaction channels themselves:

$$c_j(x) = \alpha_j(x)(w_j^\top x)(g_j^\top x).$$

From a rate-distortion perspective [PKW24], current FFNs may be hiding structure in a way that makes it difficult to disentangle without a more explicit inductive bias. By contrast, the proposed Tucker generalization would encourage the model to learn a more explicit interaction structure, which could in turn facilitate mechanistic interpretability without sacrificing the empirical advantage of multiplicative gating.

Email address: `aniket4@illinois.edu`

References

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. *Attention Is All You Need*. Advances in Neural Information Processing Systems, 2017.

[Sha20] N. Shazeer. *GLU Variants Improve Transformer*. arXiv:2002.05202, 2020.

[KB09] T. G. Kolda and B. W. Bader. *Tensor Decompositions and Applications*. SIAM Review, **51**(3):455–500, 2009.

[Nad64] E. A. Nadaraya. *On Estimating Regression*. Theory of Probability and its Applications, **9**(1):141–142, 1964.

[Pai25] D. Pai. *Regression is All You Need*. Tilde Research, 2025. https://blog.tilderesearch.com/vignettes/regression

[JSSB20] S. M. Jayakumar, J. Menick, W. M. Czarnecki, J. Schwarz, J. Rae, S. Osindero, Y. W. Teh, T. Harley, and R. Pascanu. *Multiplicative Interactions and Where to Find Them*. ICLR, 2020.

[Ols+22] C. Olsson, N. Elhage, N. Nanda, N. Joseph, N. DasSarma, T. Henighan, et al. *In-context Learning and Induction Heads*. Transformer Circuits Thread, 2022.

[Fu+23] D. Y. Fu, T. Dao, K. K. Saab, A. W. Thomas, A. Rudra, and C. Ré. *Hungry Hungry Hippos: Towards Language Modeling with State Space Models*. ICLR, 2023.

[PKW24] D. Pai, A. Keigwin, and A. Wang. *The Rate Distortion Dance of Sparse Autoencoders*. Tilde Research, 2024. https://blog.tilderesearch.com/blog/rate-distortion-saes

[Bri+23] T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, et al. *Towards Monosemanticity: Decomposing Language Models with Dictionary Learning*. Transformer Circuits Thread, 2023.